Information Security Education & Awareness (ISEA) Phase III
Tezpur University

INTERNSHIP REPORT

Assignment 7

Secure Network Application Development Using TCP

Student Name	Parinita Devi
Roll / ID No.	CS-BTC25-39
Course / Subject	Computer Networks / Network Programming
Internship Program	ISEA Phase III, Tezpur University
GitHub Repository	https://github.com/parinita-devi

Submitted as part of the ISEA Phase III Internship Program

1. Objective

- Enhance the multi-client TCP application developed in Assignment 6 by integrating core application-level security mechanisms: user authentication, password hashing, session control, and input validation.
- Emphasize practical, hands-on implementation of application-level security and its verification through live packet-level analysis using Wireshark.

2. Security Features Implemented

- **User Authentication:** Username and password combinations are validated against stored user records before granting access.
- **SHA-256 Password Hashing:** Credentials are stored as SHA-256 hashes inside users.csv instead of plain text, preventing exposure of raw passwords.
- **Duplicate Login Prevention:** Active user sessions are tracked in memory to block simultaneous logins to the same account from multiple clients.
- **Input Validation:** Empty fields, whitespace-only entries, and malformed requests are rejected before processing.
- **Failed Login Protection & Account Lockout:** Accounts are temporarily locked after five consecutive incorrect password attempts, mitigating brute-force attacks.
- **Session Management & Secure Logging:** Session state is tracked per client and operational events are written to security_log.txt, with passwords deliberately excluded from logs.

3. System Architecture & Network Setup

3.1 Network Topology

A single-switch Mininet topology was used to emulate the client-server network, created using the command below.

```
sudo mn --topo single,5
```

3.2 Node Roles

- **Server (h1 – IP: 10.0.0.1):** Listens on TCP port 12345 (alternatively 5000), and handles password hashing, authentication state, and session control.
- **Client Nodes (h2, h3 – IPs: 10.0.0.2, 10.0.0.3):** Run client_gui.py to initiate socket connections and transmit login requests to the server.

3.3 Data Flow

The client GUI sends formatted request strings, such as LOGIN|username|password, over a TCP socket connection. On receipt, the server hashes the incoming password using SHA-256 and validates the result against the stored hash in users.csv before returning an AUTH_SUCCESS or AUTH_FAILED response to the client.

4. Implementation Details

4.1 server.py

- Loads stored credentials from users.csv and hashes incoming passwords using hashlib.sha256() for comparison.
- Maintains session state using in-memory set data structures: active_users (for duplicate-login prevention) and locked_accounts (for lockout enforcement).
- Handles multiple simultaneous clients using threading.Thread, spawning a dedicated handler thread per connection.

4.2 client_gui.py

- Initializes a Tkinter-based graphical interface for username and password entry.
- Establishes a TCP connection using socket.AF_INET and socket.SOCK_STREAM, and formats the login payload before transmission.

4.3 Data & Log Files

- **users.csv:** Stores credentials in the format username,password_hash.
- **security_log.txt:** Records timestamped operational and security events (logins, failures, lockouts) without ever logging plaintext passwords.

5. Testing & Verification

The following test cases were executed on the Mininet topology to verify each security mechanism, with results captured directly from the client GUI.

Test Case 1: Successful User Authentication

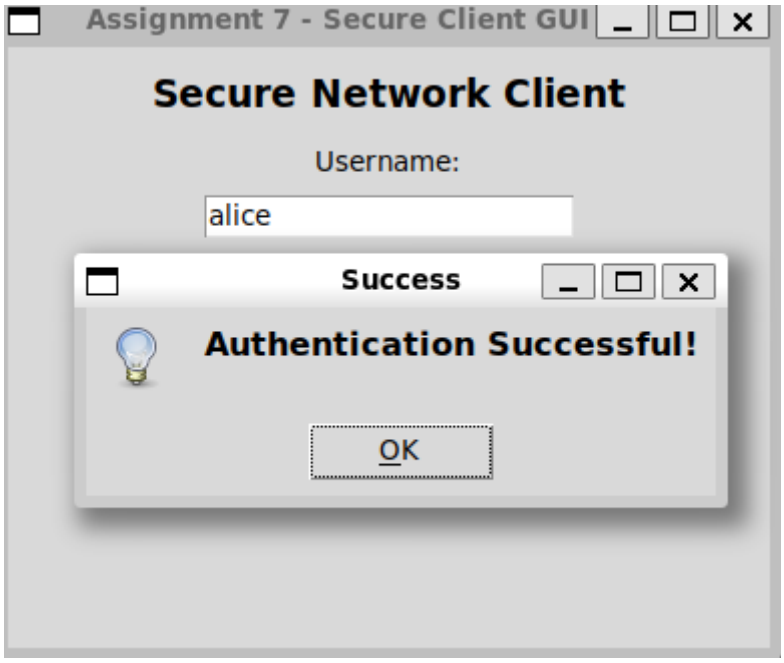

Figure 1: Successful authentication response on client node h2

The client on node h2 submitted valid credentials for user ‘alice’. The server validated the SHA-256 hash of the supplied password against the stored record in users.csv and returned an “Authentication Successful!” confirmation, demonstrating correct end-to-end authentication.

Test Case 2: Duplicate Login Prevention

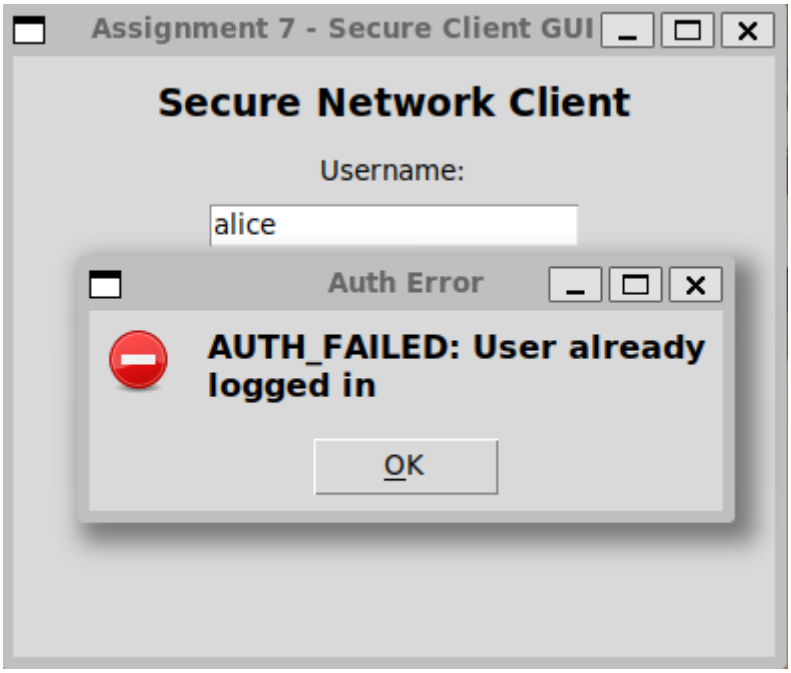

Figure 2: Duplicate login attempt blocked on client node h3

With user ‘alice’ already logged in from node h2, a second login attempt for the same account from node h3 was rejected by the server with the message “AUTH_FAILED: User already logged in”, confirming that the active_users session tracking correctly prevents simultaneous logins to a single account.

Test Case 3: Failed Login Protection & Account Lockout

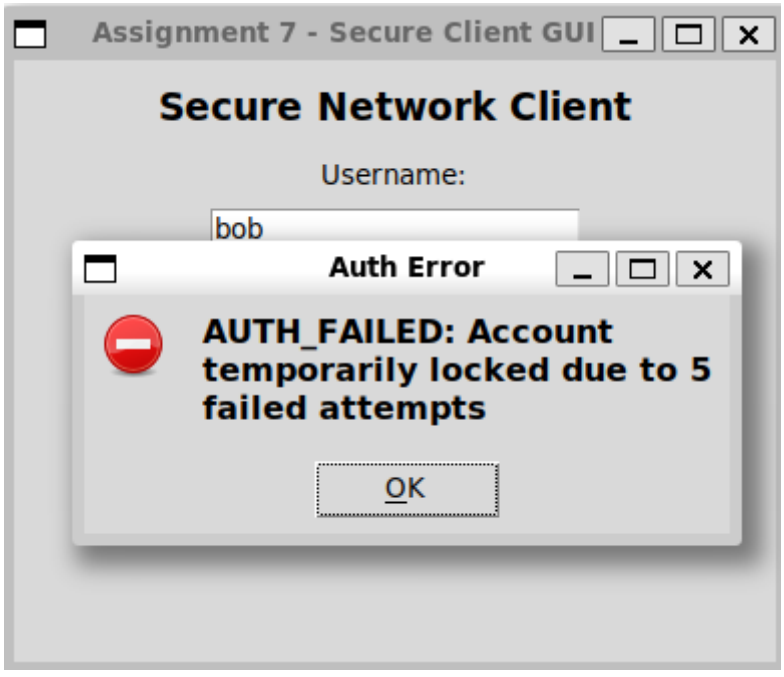

Figure 3: Account lockout triggered after repeated failed attempts

Five consecutive incorrect password submissions for user ‘bob’ caused the server to temporarily lock the account, returning “AUTH_FAILED: Account temporarily locked due to 5 failed attempts.” This confirms that the lockout safeguard is correctly enforced by the locked_accounts tracking mechanism, mitigating brute-force login attempts.

6. Wireshark Packet Verification

- **Filter Used:** tcp.port == 12345 (or tcp.port == 5000)

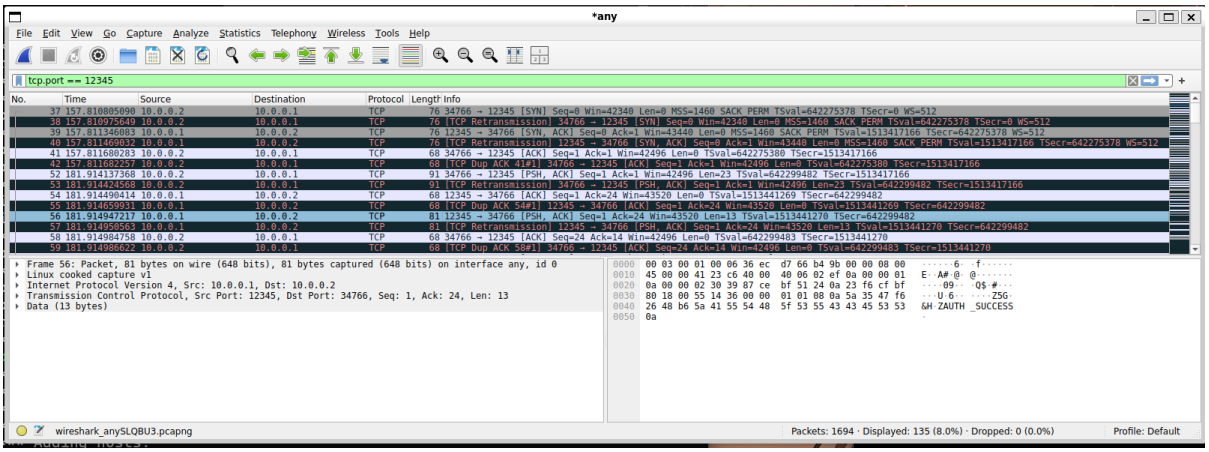

Figure 4: Wireshark capture of the TCP authentication exchange between h2 (10.0.0.2) and h1 (10.0.0.1)

The capture shows the standard TCP three-way handshake (SYN, SYN-ACK, ACK) between client 10.0.0.2 and server 10.0.0.1 on port 12345, followed by PSH, ACK segments carrying the application-layer authentication exchange. The highlighted packet (Frame 56, 13 bytes of payload) shows the server's response payload decoding to AUTH_SUCCESS, confirming that the authentication result is transmitted reliably over the established TCP connection.

7. Conclusion

Reusing the multi-client TCP application from Assignment 6 provided a stable functional base for layering critical application-level security features. The integration of SHA-256 password hashing, duplicate-login prevention, input validation, and account lockout safeguards was verified through both GUI-level test cases and Wireshark packet analysis.

Together, these mechanisms demonstrate an effective, practical approach to protecting client-server TCP traffic against basic unauthorized access, while the packet capture confirms that the authentication handshake and results are transmitted correctly and reliably across the network.